Lecture 18 · Part IV · Lecture notes

Attention and transformers

Dataset: IMDb film reviews

How to use these notes

The primary text for this lecture is Géron, Chapters 14–15. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 One change removes three problems **2**

2 Derivation: scaled dot-product attention **2**

2.1 Positions . . . 3

2.2 One block, and what masking is for . . . 3

2.3 The quadratic cost, and reading attention carefully . . . 4

3 Fine-tuning a pretrained transformer **4**

3.1 Where the pretrained model earns its place . . . 5

4 The same representation, two more deliverables **6**

5 The vectoriser fitted before the split **6**

5.1 The text leak that really is expensive . . . 7

6 Is 86.0% good? **7**

7 The notebook **8**

8 Design exercise: a social platform's comment-moderation system **8**

9 Exercises **10**

Summary **10**

---

1 One change removes three problems

Lecture 17 classified IMDb reviews: words and subword tokens, an embedding table, and a bidirectional GRU trained on logits with cross-entropy — which lost to a bag-of-words anchor. However well it is trained, a recurrent model carries three limits: the computation is sequential along the sequence; everything remembered must fit through one fixed-width state; and position 400 is four hundred steps from position 2, however good the gates.

Let every position compute a weighted sum over every position *directly*, with the weights learned from the content. No recurrence, no single state, and every pair of positions one step apart.

2 Derivation: scaled dot-product attention

The output at position i is $\sum_j w_{ij}\mathbf{v}_j$ — a weighted sum of values, one per position. Note what is already gone: there is no recurrence in that expression, so nothing forces the positions to be computed in order. Everything below is about where w_{ij} comes from.

Project each position three ways with learned matrices. A **query** is what this position is looking for; a **key** is what that position offers; a **value** is what it contributes if chosen. Score every pair by $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j$ — a dot product, because it is the cheapest similarity a matrix multiply computes for all pairs at once — then normalise along the sequence with Lecture 17's softmax and take the weighted sum. For all positions at once: $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$.

Why divide by $\sqrt{d_k}$

Take $\mathbf{q}$ and $\mathbf{k}$ to have independent components with mean 0 and variance 1. Then $s = \sum_{d=1}^{d_k} q_d k_d$ is a sum of d_k independent zero-mean terms, each of variance $\mathrm{Var}(q_d)\,\mathrm{Var}(k_d) = 1$, so

$$\mathrm{Var}(s) = d_k.$$

The scores grow with the dimension: at $d_k = 64$, typical scores are already of order 8. Softmax of large scores saturates — one weight goes to 1, the rest to 0, and its gradient goes to zero with them, which is the vanishing gradient of Lecture 16 arriving through a different door.
Dividing by $\sqrt{d_k}$ returns the variance to 1 whatever the dimension:

$$\text{Attention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\mathsf{T}}{\sqrt{d_k}}\right)\mathbf{V}.$$

The scale is not a hyperparameter. It is the number that makes the score distribution independent of d_k.

2.1 Positions

Failure condition — the problem attention creates

$\text{softmax}(\mathbf{Q}\mathbf{K}^\mathsf{T})\mathbf{V}$ is *permutation equivariant*: shuffle the input positions and the outputs shuffle with them, unchanged.
So the model literally cannot tell "the film was not good" from "good, the film was not".
Recurrence got order for free by processing in order. Attention has to be *told*.

Positional encoding adds a position-dependent vector before the first layer: *sinusoidal*, fixed and extending to unseen lengths; *learned*, an embedding table indexed by position; or *relative*, encoding the distance between two positions in the score, which is what most current models do.

2.2 One block, and what masking is for

A transformer block is self-attention, then add the input back and normalise, then a position-wise feed-forward network applied identically at every position, then add and normalise again. The residual gives the gradient an additive path through depth — the same argument as an LSTM gate, one level up — and layer normalisation, Lecture 12's batch normalisation taken over each position's features instead of over the batch, keeps the variance from drifting as blocks stack. *A transformer is deep, and it is deep only because of these two.*

A decoder generating text must not look at what it has not written, so $s_{ij} = -\infty$ for $j > i$ before the softmax. $e^{-\infty} = 0$, so those positions get exactly zero weight and contribute nothing — and, crucially, no gradient either. That is Lecture 15's causality condition again, enforced in the architecture rather than in the data split: *an unmasked decoder is a model that has read the answer.*

Attention itself is six lines of code, and every one is a step of the derivation:

```
def attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)   # the derivation
    if mask is not None:
        scores = scores.masked_fill(mask, float("-inf"))
    w = scores.softmax(dim=-1)
    return w @ V, w
```

`mask` is `True` where $j > i$, the positions a decoder must not see. The rest of a transformer is arrangement around that function. Which is worth saying plainly: *the architecture that changed the field is, at its centre, a similarity, a softmax and a weighted average.*

2.3 The quadratic cost, and reading attention carefully

$\mathbf{QK}^\top$ is $T \times T$: at $T = 512$ that is 262,144 scores, at $T = 8{,}192$ it is 67 million. Three things are done about it: *sparse patterns* (a local window plus a few global positions, making the cost linear); *low-rank or kernel approximations* (approximate the softmax so the product is never formed); and *better implementations* — the exact same computation, tiled so it never writes the $T \times T$ matrix to memory, which is what most systems actually use. The first two change the mathematics and the third does not, and the third moved the practical limit further than either.

Failure condition — a context length that runs out of memory

Attention is quadratic in the sequence length, so doubling the context quadruples the score matrix. The failure arrives as an out-of-memory error at a length that worked on the previous run, and no accuracy number anywhere hints at how close you were.

Attention weights are easy to plot and easy to over-read. They are *not an explanation*: a weight says this value contributed to that output, not that the decision depended on it — the residual stream carries information around the attention, and different weight patterns can give identical outputs. Useful for forming a hypothesis; not evidence on its own. The test is the one used everywhere else in this course: remove the thing and measure whether the number moves.

3 Fine-tuning a pretrained transformer

The model we borrow has 66,955,010 parameters, the same 30,522-token vocabulary as last lecture, an embedding width of 768, and was pretrained to predict masked words. *Last lecture we took* $30{,}522 \times 768$ *numbers out of this model and threw the rest away. The rest is where the language is.*

Fine-tuning puts a new, randomly initialised head on a pretrained body (`AutoModelForSequenceClassification` with `num_labels=2`; every weight pretrained except the head), trained with AdamW — Adam with weight decay, a small pull of every weight towards zero at each step — at lr $= 2 \times 10^{-5}$ — fifty times smaller than the 10^{-3} used from scratch, because large steps destroy what was pretrained — for one pass over the data rather than two. The loss is still `nn.CrossEntropyLoss`, logits in, as always.

Setting	Value
Reviews used to fine-tune	2,000
Epochs	1
Batch size, learning rate	16, 0.00002
Test reviews scored, sampled at random	3,000

Run in float32, and sized so that the notebook runs on a free CPU. One pass, fixed in advance: with more budget the number of epochs is a hyperparameter, chosen on the validation split — never on the test set.

How much is the head, and how much is the body?

The same backbone with its untrained head and no fine-tuning scores **49.9%**, against 50.0% for always predicting one class, on 3,000 test reviews of which 50.1% are positive.
Note it is *no better* than the majority class: an untrained head is an arbitrary hyperplane and which side it puts each class on is a coin toss. The pretraining on its own predicts nothing about sentiment, because nothing has told it what the two output units mean.

Model	Test accuracy
Always one class	50.0%
tf-idf + logistic regression, the same 2,000 reviews	84.9%
GRU from scratch	63.0%
DistilBERT, fine-tuned	86.0%

22.97 points over the model built last lecture — and only 1.07 points over a bag of words fitted on the same 2,000 labels, which is the second result of the afternoon and the less comfortable one. Every row but the first is scored on the same 3,000 test reviews.

Say it in errors, not in points

The GRU misclassifies 1,109 reviews out of 3,000; the fine-tuned transformer 420. *62% of the mistakes removed.*
Percentage points flatter a model near the top of the range. The error count is what the desk cares about, because each one is a review someone has to fix by hand.

3.1 Where the pretrained model earns its place

The comparison above is at one label budget: 2,000 reviews for each model, and there the transformer leads the bag of words by 1.07 points. Pretraining is knowledge the labels do not have to supply, so the expectation is that its lead *grows* as labels get scarcer and shrinks as they arrive — and every real feedback desk starts with few labels. The notebook measures one point on that curve, not the curve.

Measuring the curve means repeating the comparison at several label budgets, from a couple of hundred reviews to several thousand, on one fixed test sample drawn at random, with each model's budget — optimisation steps for the transformer, regularisation strength for the linear model — chosen on the validation split at every size. Fixing the epochs instead starves the smallest run of steps, and fixing the steps undertrains the largest; the two protocols can disagree

about the *sign* of the gap. That is several fine-tunes, more than a free CPU runs inside a lecture, so until it is run *"the pretraining is worth thousands of labels" is a hypothesis*, not yet a number to quote to whoever is paying for the annotation.

4 The same representation, two more deliverables

The desk wanted two more things — "find me the ones like this one" and "tell me what people keep complaining about". Neither is classification, and both fall out of the same pretrained models, because what those models produce is a *representation*, and a label was only ever one thing to do with it.

Mean-pooling over the *real* tokens gives one vector per review — and that pooling is last lecture's padding bug in its other costume: divide by the padded length instead of the true one and every short review is scaled towards zero. Normalising onto the unit sphere makes the dot product the cosine.

Searching 2,000 negative reviews, embedded once at 384 dimensions, is then one matrix product — no index, no training, no labels.

For the query "the acting was wooden and unconvincing", the embedding's nearest review (cosine 0.555) opens "Ed Wood rides again", and keyword search's (tf-idf cosine 0.194) opens "This film was sheer boredom" — two different reviews, neither opening with the query's words. The two scores live in different spaces and are not comparable, and nothing here says which list is *better*: that takes labelled queries and a ranking metric, which is where the next lecture starts.

Clustering the same vectors is Lecture 6 unchanged, with k chosen by silhouette rather than by eye. Over the 2,000 reviews, eight groups, at a silhouette of 0.0167 — low in absolute terms, because complaint text does not fall into well-separated balls. The largest is legible: 438 reviews around *bad, worst, terrible, seen*. The next two are 373 around *flynn, actor, ed, bishop* and 346 around *horror, zombie, cave, zombies* — the first of those named by proper nouns, a handful of films reviewed many times, which is a topic rather than a complaint. Grouping finds what recurs; deciding which recurrences the desk cares about is still a person's job. Those terms are not the cluster centres — they are the words whose average tf-idf weight is highest inside the group relative to outside it, because *a group is named by what makes it different, not by what is common everywhere*.

5 The vectoriser fitted before the split

The natural way to write Lecture 17's anchor — a pipeline that vectorises the reviews with tf-idf, classifies them with logistic regression, and reports the test accuracy — has the same gap as a scaler fitted before the split (Lecture 3), and this time the object being fitted is not a scaler. Call `fit_transform` on *every* document and then split, and the code runs, splits, never fits the classifier on test rows, and reports a plausible number.

The same experiment at two corpus sizes: the whole of IMDb over 3 seeds, and 400 documents — the size of a real feedback corpus in its first month — over 20 seeds. Leaky and honest run on identical splits with identical seeds, so the difference is the leak and nothing else. Plot every seed

and read each row's spread before its mean: on 25,000 reviews the leak is small and steady; on 400 the seeds fall on both sides of zero, and the mean gap of 0.30 points is lost inside that scatter. *This is a null result.*

Three leaks, and none of them loud

A scaler fitted before the split (Lecture 3) raises no error and makes the score optimistic by an amount the code does not tell you; Lecture 8's per-batch mean was worth 0.067 points; today's vectoriser leak is worth nothing you can measure. A leaked score and an honest score can be identical, so you cannot tell from the number which one you are holding. The procedure is what separates them, and the procedure is what gets checked in review.

5.1 The text leak that really is expensive

The same document on both sides of the split. No pipeline protects you: every object is fitted correctly, on training rows only, and the test row is still one the model has memorised. On scraped or user-submitted text it is the normal case — a resubmit button is enough.

Three lines find them: normalise case and whitespace, and count the test documents whose normalised text is also in training. To price the leak, build a corpus where it is common: reviews of which three in ten were submitted twice. Then split two ways and fit the same honest pipeline inside each: at random over rows, and grouped by entry, so that every copy of an entry falls wholly on one side. The random split reads 7.83 points higher, and nothing was fitted on the test set in either case — *the difference is entirely which rows the split separated*. The rows split wrongly cost far more than the object fitted wrongly.

Failure	Diagnosed in	Measured cost
Scaler fitted before the split	Lecture 3	an optimistic score, no error
Accuracy under class imbalance	Lecture 3	a useless model above 90%
Missing `zero_grad()`	Lecture 8	66 points
Random split on a time series	Lecture 15	skill overstated by 9.0%
Summarising at the padded last position	Lecture 17	the state after the padding, no error
Softmax before the loss	Lecture 17	the wrong gradient, no error
Vectoriser fitted before the split	today	0.30 points, not distinguishable from zero
Duplicate documents across the split	today	7.83 points

6 Is 86.0% good?

Against the desk's own criterion: 420 reviews out of the 3,000 scored are routed wrongly and nobody sees them again. Whether that is acceptable depends on what happens to a missed complaint, and we have not asked.

What a defensible report says

Quote the error count, not the percentage. Say which reviews are misread — short ones, sarcastic ones, ones whose verdict is in the last clause. Offer an abstain band around a probability of one half, and let the desk choose how wide it is.
The model is not the deliverable. The routing decision is.

7 The notebook

The notebook rebuilds Lecture 17's GRU and scores it, DistilBERT before and after one pass of fine-tuning, and a bag of words fitted on the same 2,000 labels, all on the same 3,000 test reviews. It then embeds 2,000 negative reviews for search and grouping, and measures the vectoriser leak and the duplicate leak. It runs on a free CPU.

What to change

Fine-tune at lr = 10^{-3}, the from-scratch model's rate, and re-run. The loss rises within twenty steps and never recovers: a pretrained body is destroyed in tens of steps, not thousands.

8 Design exercise: a social platform's comment-moderation system

The lecture closes with a design exercise in the manner of an ML system-design interview, framed as the ML system behind a company's product.

The task. A social platform's comments are where its users talk and where its advertisers sit. Its moderation system must decide, at the moment a comment is posted, whether it is toxic — harassment, threats, slurs — and output a probability, so that the comment is posted, held for review, or removed. The decision has to be made before the page refreshes, and every error is read by someone: a missed threat by its target, a wrongly held comment by its author. The business measures retention, the review desk's cost, and the fines for a missed removal or a denied appeal. Given: comments labelled by the moderation team — tens of thousands, out of the millions posted — and its written policy. Millions of comments a day, of which a few in a hundred are toxic; a decision in tens of milliseconds. The output feeds the posting pipeline, the human-review queue, the appeals process and the transparency report the regulator reads.

1. **Frame it.** The model outputs $p(\text{toxic} \mid \text{text})$; the decision — post, hold, remove — is a threshold on it, and the threshold belongs to the trust-and-safety team. A few in a hundred are toxic, so the anchor "always post" is right almost always; report both errors as counts, toxic comments posted and clean comments held, beside the desk's capacity, since every held comment is a reviewer's time and a user waiting. *Pitfall:* accuracy, and a threshold of one half because that is what `argmax` gives; under this imbalance accuracy cannot see the toxic class, and one half is a policy chosen by nobody. Sweep the threshold, report the error counts in both directions, and let the team choose the operating point.
2. **The label.** A moderator's decision under the written policy; moderators disagree, so have

more than one read a slice and record the disagreement, which is the ceiling on any model. Quoted slurs, sarcasm and reclaimed words are where the labels are noisiest. *Pitfall:* labelling only what was reported or the old filter caught, so that every comment never reviewed is a negative that may not be one, the model learns the old filter's blind spots as truth, and the users it failed stay failed. Label a random sample of all posted comments as well, and report every number on that sample.

3. **Split.** Train on earlier comments, validate on later ones, test on the latest, since slang and coded words move by the month; choose the threshold, the passes and the model on validation and read the test comments once. *Pitfall:* the same comment on both sides of the split — the same text pasted into fifty threads is the normal case for abuse. No pipeline protects you: every object is fitted correctly, and the test row is still one the model memorised, so the launch score is one no user sees. Normalise case and whitespace, group identical comments, split by time, and count the duplicates across the cut.

4. **The anchor.** tf-idf over words and adjacent pairs with logistic regression, trained on the same labels the transformer will get, its regularisation strength chosen on the validation comments and its scoring time per comment written down, since posting has a latency budget the metric cannot see. *Pitfall:* `fit_transform` on every comment and then the split. It runs, never fits the classifier on test rows, and reports a plausible number; the measured cost is small, and that is not the point, because a leaked score and an honest one can be identical and the procedure is what is reviewed before release. The vectoriser is fitted on training comments, inside a pipeline.

5. **Fine-tune.** A pretrained transformer with a randomly initialised two-logit head, every weight trained at a learning rate tens of times smaller than from scratch, cross-entropy on the logits. Few labels is where pretraining earns its place; comments are short, so the quadratic cost is paid on a short context, and the per-comment time is measured against the posting pipeline's budget. *Pitfall:* the from-scratch learning rate. The loss rises within tens of steps and never recovers, and what remains scores like a random head — the majority class, which posts everything. Small steps, few passes, and the number of passes chosen on validation.

6. **Pooling.** Batches are padded to the longest comment; the mask keeps the padding out of the attention scores, and the pooled vector must average the real tokens only. The same vector finds near-duplicates of a removed comment and groups what users keep reporting. *Pitfall:* dividing by the padded length instead of the mask's sum. Every short comment is scaled towards zero, and short comments are where the insults are. Divide by the number of real tokens, and assert that padding the same comment two ways gives the same vector.

7. **Compare fairly.** Both models on the same labels and the same test comments, scored by one function, in errors rather than points; each model's budget — steps for the transformer, regularisation for the linear model — chosen on validation at every label size, and the comparison repeated at several sizes, since the question the labelling budget pays to answer is how many labels the pretraining is worth. *Pitfall:* the transformer's passes tuned and the anchor's regularisation left at its default. The lead over a bag of words on the same labels is small enough that the tuning can be all of it; tune both on the same split.

8. **In production.** Three bands on the probability — post, hold, remove — with the hold band sized to the desk's capacity; appeals and overturns flow back as labels. Release to a share of traffic first, the old filter on the rest, and compare hold rate, overturn rate and latency across

the two; retrain on a recent window, as coded words appear monthly, with the model team and the desk sharing on-call; review a random slice of posts regardless of score, because the model changes what gets reported. *Pitfall:* false positives concentrated on a dialect or a community, invisible in the mean, because the labels marked words toxic wherever they appeared — users who leave, and a case the regulator opens. Count the errors per segment before launch, route those segments to the human band rather than to removal, and read the overturns per segment as the live measure.

9. **Ship it.** Moderator labels plus a random sample of everything posted → duplicates grouped, split by time → bag-of-words anchor, vectoriser fitted on training comments, regularisation tuned → a pretrained transformer fine-tuned at a small learning rate, pooled over real tokens → both compared in error counts at the same label budget → thresholds set by the team → a share of traffic first; post, hold, remove; appeals as labels; per-segment monitoring → retrain.

9 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 19's deck and in the exercise book.

1. Scaled dot-product attention divides by $\sqrt{d_k}$. State what the variance of the unscaled dot product is, and what the softmax does without the division. [5]

2. A decoder must not attend to positions after the one it is predicting. State the mechanism that enforces it, and what would leak without it. [5]

3. A pretrained body is fine-tuned at 10^{-3}, the rate that worked for the from-scratch model. Predict what happens, and state the rate the lecture uses instead. [4]

4. A tf-idf vectoriser is fitted on every document of a 400-review corpus, and the matrix is split afterwards. Measured, the leak is worth 0.30 points. Say where the vectoriser should be fitted, and why the small number is no reason to leave the code as it is. [4]

5. A corpus contains duplicate documents across the train/test split. State why no pipeline fixes it, and say how the split must assign the rows instead. [4]

Summary

Attention is three projections, a scaled dot product, a softmax and a weighted sum — and the scale is not a hyperparameter: the score variance is d_k, so dividing by $\sqrt{d_k}$ is what keeps the softmax off its saturated tails whatever the width. Attention is permutation equivariant, so position has to be added; a decoder is made causal by a mask; and the cost is quadratic in the sequence length.

Fine-tuning a 67-million-parameter pretrained body on 2,000 reviews reached 86.0%: 22.97 points over the recurrent model, 62% of its errors removed — and only 1.07 points over tf-idf with logistic regression fitted on the same labels. Whether the pretraining's lead grows as labels get scarcer is the experiment to run next, not a result here. The same family of models gives one vector per review, and with it search and grouping without a single label.

Two leaks, measured against each other. A vectoriser fitted before the split costs 0.30 points on a small corpus, inside the seed-to-seed scatter — a null result, and the rule stays procedural anyway. Duplicate documents across the split cost 7.83 points, with nothing fitted wrongly at all. The closing design exercise built a social platform's comment-moderation system in nine steps — a threshold that is policy, labels beyond what the old filter caught, duplicates kept on one side of a split by time, a vectoriser fitted where it should be, a body fine-tuned at a small rate, pooling over real tokens, both models tuned on the same split, a share of traffic first, and the errors counted per dialect before launch.